

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I (Ch. Sri Lakshmi/192311342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch. Sri Lakshmi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS

1: HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header

Step 1: Read each HTTP response header field.

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Step 2: Check whether every field follows HTTP standards.

- HTTP Version → Correct
- Status Code (200 OK) → Correct
- Date → Correct format
- Content-Type → Correct

- **Content-Length** → **Incorrect** because it is **-150 (negative value)**
- **Connection** → **Correct**

Answer: The invalid field is **Content-Length: -150**, since Content-Length cannot be negative.

2. Analyze the issue with Content-Length and explain its impact

Step 1: Content-Length tells the browser how many bytes are present in the response body.

Step 2: The value must always be **0 or a positive integer**.

Step 3: Since **-150** is invalid, the browser cannot determine the actual size of the content.

Impact:

- Browser may reject the response.
- Page may not load correctly.
- Connection may terminate unexpectedly.
- Some browsers may ignore the response.

3. Determine whether the status code matches the response correctness

Step 1: Status code **200 OK** means the request was processed successfully.

Step 2: However, the response contains an invalid header (negative Content-Length).

Step 3: Therefore, the response is not completely correct.

Answer:

The **200 OK** status does **not fully match** the response because an invalid header is present.

4. Suggest corrections for the identified errors

Step 1: Replace the negative Content-Length with the correct size.

Example:

Content-Length: 150

Step 2: Ensure all headers follow HTTP standards.

Correct Header

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

2: DOM Structure Analysis

1. Identify missing or improperly closed elements

Given code:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Step 1: Check opening and closing tags.

Step 2: Observe that the `<p>` tag is not closed.

Answer:

Missing closing tag:

```
</p>
```

2. Analyze how the browser will interpret this structure

Step 1: Modern browsers automatically fix minor HTML errors.

Step 2: The browser automatically inserts the missing `</p>` before `</div>`.

Answer:

The browser creates a corrected DOM automatically and continues rendering the page.

3. Determine the impact on rendering and layout

Step 1: Browser displays the text normally.

Step 2: Invalid HTML may produce inconsistent layouts in different browsers.

Impact

- Unexpected formatting
- CSS may behave differently
- Difficult debugging

4. Suggest corrections to make the DOM valid

Correct code:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```

3: GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST

Step 1: GET sends data in the URL.

Example:

/login?user=admin&pass=1234

Step 2: POST sends data inside the request body.

Difference

GET	POST
Data in URL	Data in request body
Visible	Hidden from URL

GET	POST
Less secure	More secure

2. Analyze which method is more suitable for login and why

Step 1: Login contains sensitive information.

Step 2: Password should never appear in the URL.

Step 3: POST hides credentials in the request body.

Answer:

POST is more suitable because it protects login credentials better than GET.

3. Identify security risks in using GET for authentication

Step 1: Password appears in browser history.

Step 2: Password is stored in server logs.

Step 3: URL can be bookmarked or shared.

Risks

- Password exposure
- Privacy issues
- Easy credential leakage

4. Suggest best practices for secure login implementation

- Use POST method.
- Use HTTPS.
- Encrypt passwords.
- Store hashed passwords.
- Use secure session management.

4: HTML Code Inspection

1. Identify missing closing tags

Given code:

```
<h1>Welcome
```

```
<p>This is a paragraph
```

```

```

Missing closing tags are:

```
</h1>
```

```
</p>
```

2. Analyze issues with the tag

Step 1: Image tag contains only the source.

Step 2: It should also include an **alt** attribute.

Example:

```

```

3. Determine how browsers will handle these errors

Step 1: Browser automatically closes missing tags.

Step 2: Page usually displays correctly.

Step 3: However, HTML becomes invalid and may affect CSS or JavaScript.

4. Suggest a corrected version of the code

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Sample</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome</h1>
```

```
<p>This is a paragraph.</p>
```

```

</body>
</html>
```

5: Browser–Server Communication Flow

1. Interpret each step in the communication flow

Step 1: User enters a URL.

Browser receives the website address.

Step 2: DNS Lookup.

Domain name is converted into an IP address.

Step 3: HTTP Request.

Browser sends a request to the web server.

Step 4: Server Processing.

Server processes the request and prepares the response.

Step 5: HTTP Response.

Server sends HTML, CSS, JavaScript, and other resources.

Step 6: Browser Rendering.

Browser displays the webpage to the user.

2. Identify where delays can occur

Delays may happen during:

- DNS lookup
- Network transmission
- Server processing
- Large response download
- Browser rendering

3. Analyze the role of DNS in the process

Step 1: DNS translates the website name into an IP address.

Example:

www.google.com

↓

142.250.xx.xx

Step 2: Without DNS, the browser cannot locate the server.

Answer:

DNS acts as the **directory service of the Internet**, helping browsers find the correct server.

4. Suggest optimizations to improve performance

- Use DNS caching.
- Enable browser caching.
- Compress files using GZIP.
- Minify HTML, CSS, and JavaScript.
- Use a Content Delivery Network (CDN).
- Optimize server response time.